

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«Санкт-Петербургский национальный исследовательский университет ИТМО»

ФАКУЛЬТЕТ ПРОГРАММНОЙ ИНЖЕНЕРИИ И КОМПЬЮТЕРНОЙ ТЕХНИКИ

ЛАБОРАТОРНАЯ РАБОТА №2
по дисциплине
«Операционные системы»

Базовый трек
Вариант: Windows, LFU, ema-search-string

Выполнил:

Студент группы Р3316
Колбасин Владислав Ильич

Преподаватель:

Гиниятуллин Арслан

Оглавление

Задание.....	3
Исходный код.....	5
Сравнение	6
Вывод	7

Задание

Для оптимизации работы с блочными устройствами в ОС существует кэш страниц с данными, которыми мы производим операции чтения и записи на диск. Такой кэш позволяет избежать высоких задержек при повторном доступе к данным, так как операция будет выполнена с данными в RAM, а не на диске (вспомним пирамиду памяти).

В данной лабораторной работе необходимо реализовать блочный кэш в пространстве пользователя в виде динамической библиотеки (dll или so). Политику вытеснения страниц и другие элементы задания необходимо получить у преподавателя.

При выполнении работы необходимо реализовать простой API для работы с файлами, предоставляющий пользователю следующие возможности:

1. Открытие файла по заданному пути файла, доступного для чтения. Процедура возвращает некоторый хэндл на файл. Пример:
`int lab2_open(const char *path).`
2. Закрытие файла по хэндлу. Пример:
`int lab2_close(int fd).`
3. Чтение данных из файла. Пример:
`ssize_t lab2_read(int fd, void buf[.count], size_t count).`
4. Запись данных в файл. Пример:
`ssize_t lab2_write(int fd, const void buf[.count], size_t count).`
5. Перестановка позиции указателя на данные файла. Достаточно поддерживать только абсолютные координаты. Пример:
`off_t lab2_lseek(int fd, off_t offset, int whence).`
6. Синхронизация данных из кэша с диском. Пример:
`int lab2_fsync(int fd).`

Операции с диском разработанного блочного кэша должны производиться в обход page cache используемой ОС.

В рамках проверки работоспособности разработанного блочного кэша необходимо адаптировать указанную преподавателем программу-загрузчик из ЛР 1, добавив использование кэша. Запустите программу и убедитесь, что она корректно работает. Сравните производительность до и после.

Ограничения

- Программа (комплекс программ) должна быть реализован на языке C или C++.
- Если по выданному варианту задана политика вытеснения Optimal, то необходимо предоставить пользователю возможность подсказать page cache, когда будет совершен следующий доступ к данным. Это можно сделать либо добавив параметр в процедуры read и write (например, `ssize_t lab2_read(int fd, void buf[.count], size_t count, access_hint_t hint)`), либо добавив еще одну функцию

в API (например, `int lab2_advice(int fd, off_t offset, access_hint_t hint)`). `access_hint_t` в данном случае – это абсолютное время или временной интервал, по которому разработанное API будет определять время последующего доступа к данным.

- Запрещено использовать высокоуровневые абстракции над системными вызовами. Необходимо использовать, в случае Unix, процедуры `libc`.

Требования к отчету и защите

Отчет должен содержать:

титульный лист с указанием номера и названия ЛР, вашего ФИО, ФИО преподавателя практики, номера вашей группы, варианта ЛР;

текст задания в соответствии с вариантом;

краткий обзор кода;

данные о работе программы-нагрузчика до и после внедрения своего page cache;

заключение с анализом результатов и выводом

Темы для подготовки

Файловые системы в Linux, структура файла, взаимодействие ФС и диска;

Файловая система `ext4`;

Page cache в ОС;

Политики вытеснения кэшей;

Динамические библиотеки в Linux;

Исходный код

<https://github.com/secs-dev-os-course/diy-page-cache-Verteplushka/tree/lab-2>

Библиотека `cache.h` реализует кэширование файловых данных с политикой вытеснения блоков LFU, что позволяет уменьшить количество операций чтения/записи с диска.

Основные компоненты

1. Конструктор:

- `Cache(size_t blockSize, size_t maxBlocks);`
 - Конфигурирует размер блока кеша и максимальное количество блоков.

2. Открытие и закрытие файлов:

- `HANDLE openFile(const std::string& path);`
 - Открывает файл и возвращает дескриптор файла (HANDLE).
- `int closeFile(HANDLE fd);`
 - Закрывает файл и записывает все изменения обратно на диск.

3. Чтение и запись:

- `ssize_t readFile(HANDLE fd, void* buf, size_t count);`
 - Читает данные из файла, используя кешированные блоки.
- `ssize_t writeFile(HANDLE fd, const void* buf, size_t count);`
 - Записывает данные в кешированные блоки с отложенной записью на диск.

4. Перемещение указателя:

- `off_t seekFile(HANDLE fd, off_t offset, int whence);`
 - Управляет положением указателя чтения/записи в файле.

5. Синхронизация:

- `int syncFile(HANDLE fd);`
 - Принудительно записывает все изменения для указанного файла.

Сравнение

Пришлось уменьшить размер файла для чтения из первой лабораторной, чтобы запустить программу много раз на одних данных и продемонстрировать преимущества кэша. И, действительно, программа с кэшем почти в 4 раза быстрее программы без кэша на дистанции нескольких тысяч повторений.

```
kolba@DESKTOP-RRUI88Q MINGW64 /d/ITMO/year3/OS/lab2/diy-page-cache-Verteplushka/build (lab-2)
$ ./loader/Debug/emaSearchStringCacheApp.exe 3000
EMA Search: Found 3000 occurrences in 3000 repetitions, taking 13.5642 seconds.

kolba@DESKTOP-RRUI88Q MINGW64 /d/ITMO/year3/OS/lab2/diy-page-cache-Verteplushka/build (lab-2)
$ ./loader/Debug/emaSearchStringApp.exe 3000
EMA Search: Found 3000 occurrences in 3000 repetitions, taking 47.5695 seconds.
```

Однако, например при запуске этих же программ на исходном файле из первой лабораторной в 256 Мб и малом числе повторений, напротив, программа с кэшем проигрывала по времени почти в два раза. Это доказывает, что затраты на организацию кэша имеют смысл при частом обращении к одним и тем же данным, в противном случае программа тратит лишнее время на сохранение данных в память, которые повторно не используются.

Вывод

В ходе выполнения лабораторной работы был реализован блочный кэш в пространстве пользователя в виде динамической библиотеки, обеспечивающий работу с файлами. Разработанное API позволяет выполнять операции открытия, чтения, записи, синхронизации и закрытия файлов с использованием кэша, управляемого на основе заданной политики вытеснения страниц (LFU). Реализация гарантирует эффективность ввода-вывода благодаря работе с данными через RAM, минимизируя обращения к диску. Корректность и производительность решения были проверены путем адаптации программы-загрузчика, продемонстрировав заметное улучшение времени выполнения операций при многократном доступе к одним и тем же данным.